

CloudVisor: 利用嵌套虚拟化对多租户云中的虚拟机进行保护

CloudVisor: Retrofitting Protection of Virtual Machines in Multi-tenant Cloud with Nested Virtualization

背景

- 多租户云(Multi-tenant Cloud)面临的安全威胁
 - 用户的敏感数据放在第三方云平台上计算
- 面临安全问题的原因
 - off-the-shelf virtualized方式的TCB过大，上百万行代码
 - 使用同一个云服务平台的其他用户，甚至云服务商本身也是潜在的安全威胁

背景

- 前人的工作
 - 完全移除虚拟化层
 - 建立一个新的类似VMM的微内核
 - 保护VMM控制流的完整性
- 问题
 - 只考虑到其他恶意VM，没有考虑到来自VMM的安全威胁
 - 对VMM的核心代码进行改动，甚至重构VMM

设计目标

- **将安全性保护和资源管理分离**
 - 使用嵌套虚拟化技术，将安全性保护和资源管理分离
 - 和商用虚拟化栈向后兼容
 - TCBsize从上百万行代码减少到几千行
- **更全面的保护**
 - 提供对整个虚拟机的保护，保护来自恶意VM和management VM的攻击
- **性能开销低**
 - 利用虚拟化硬件支持的实现
 - 性能开销低

CloudVisor 威胁模型

- 攻击者包括本地攻击者和远程攻击者，假设他们能够控制整个VM management stack；VM的内存，磁盘和代码都有可能被泄露。
- 不会出现内部物理攻击，外部磁盘存储不可信
- 保证除 VM 自身以外的所有访问者（如 VMM、其他 VM）不能直接访问 VM 数据，如 DMA、内存转储和 I/O 数据，只能看到加密版本。
- VMM 不能伪造执行上下文来让 VM 运行。攻击者不能更改启动环境或欺骗租户的 VM 以错误的执行模式运行

CloudVisor 威胁模型

做不到什么：

- CloudVisor cannot guarantee availability and execution correctness of a tenant's VM.
- CloudVisor does not guard against side-channel attacks in the cloud, which may be hard to deploy and have very limited bandwidth to leak information.
- CloudVisor also provides no protection to interactions of a VM with its outside environments.

CloudVisor 整体设计

Design Consideration

- Transparent Interposition using Nested Virtualization
- VM-based Memory Ownership Tracking
- I/O Protection through Encryption
- Late Launch to Reduce CloudVisor Complexity

The CloudVisor Architecture

- Runs at the **most privileged level**
- commodity VMM is deprivileged into the **less privileged** mode together with the control VM and guest VMs
- CloudVisor enforces the **isolation and protection** of resources used by each guest VM and ensures the isolation among the VMM and its guest VMs
- Traditional virtualization functionalities are still done by the VMM

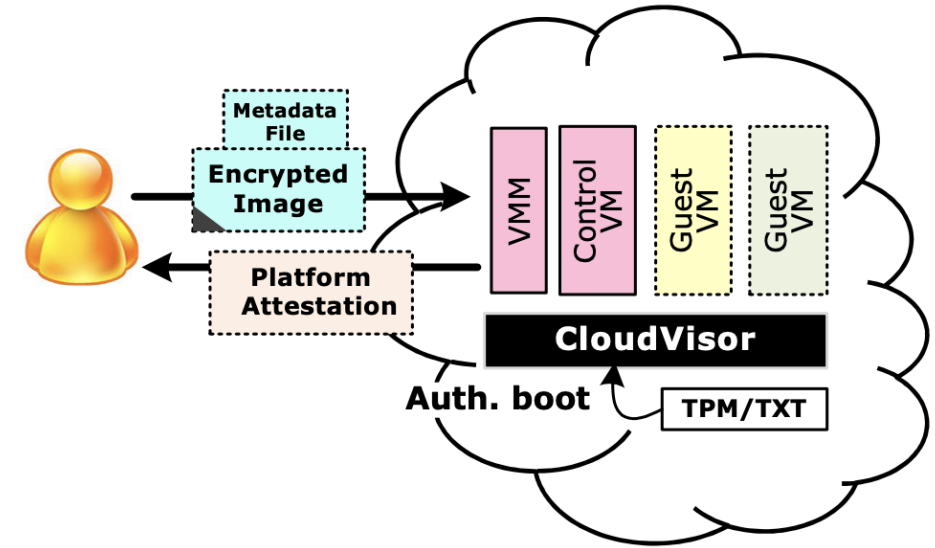


Figure 2: Overall architecture of CloudVisor.

Securing Control Transition with Nested Virtualization

- placing the VMM into a less privileged mode
- CloudVisor ensure strict isolation among the VMM and its VMs

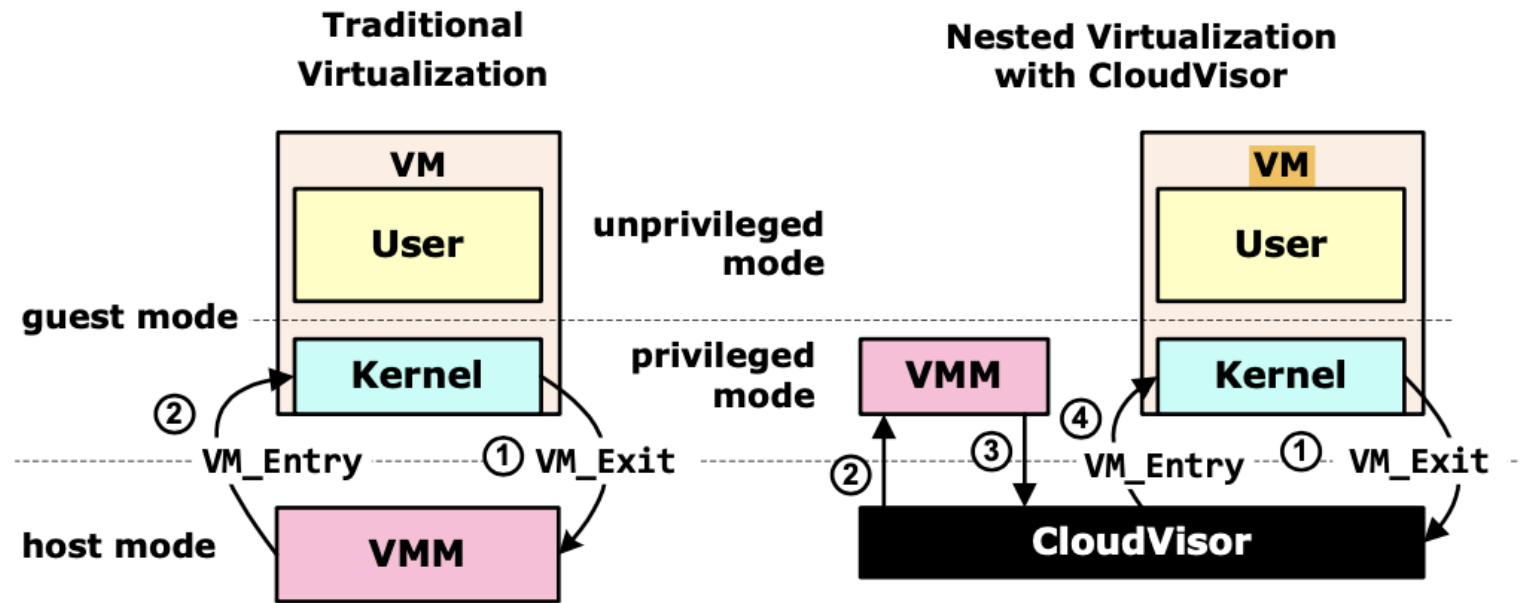


Figure 3: The general architecture of hardware-assisted virtualization (left) and how CloudVisor leverages it to secure control transfer using nested virtualization.

Enabling Interposition

- maintains a VMCS for the VMM, VMM only gets trapped on three types of architectural events relating to resource isolation:
 - **NPT/EPT faults**, which are caused by faults on translation from guest physical address to host physical address
 - Execution of **instructions in the virtualization instruction** set such as VMRead/VMWrite
 - **IOMMU faults**, which is caused by faults during the translation from device address to host physical address
- Other architectural events do not cause traps to CloudVisor and are directly delivered to the VMM.
- Change VMCS of guest VM. CloudVisor records the entry address of VM-exit in VMCS and replaces it with the entry address of the handler in CloudVisor.

Securing Control Transition

CloudVisor interposes between guest VMs and the VMM on VM exit for mainly three purposes:

- protecting CPU register contexts when a VM is interrupted;
- manipulating address translation to enforce memory isolation (detailed in section 5);
- intercepting and parsing I/O instructions to determine the I/O buffer addresses in a VM (detailed in section 6).

流程

- interposes each VM exit event (step 1), protects CPU contexts and parses I/O instructions if necessary
- forwards the VM exit event to the VMM (step 2).
- intercepts the VM entry request from the VMM (step 3)
- restores CPU contexts and resumes the execution of guest VM (step 4) accordingly

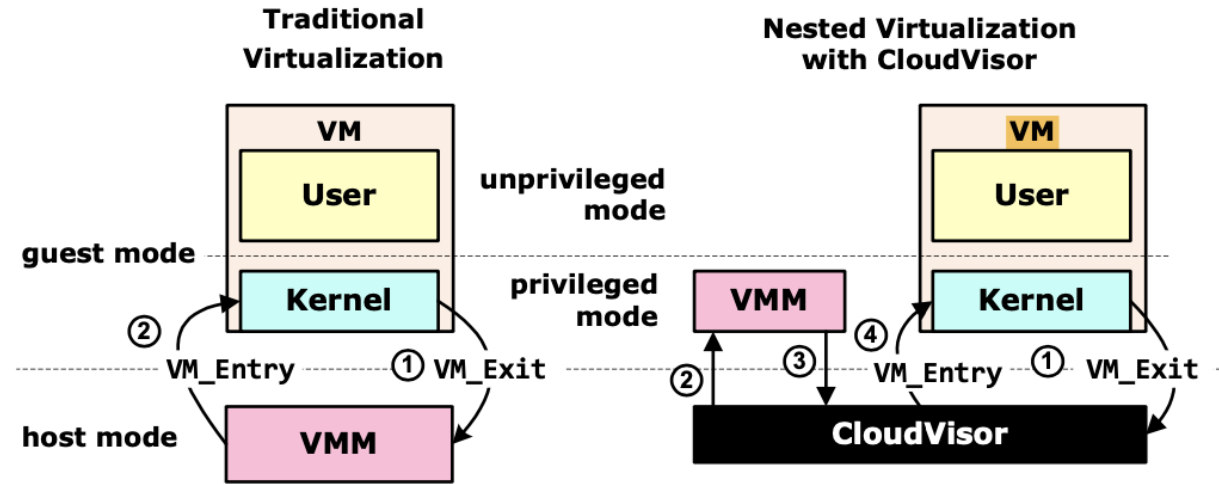


Figure 3: The general architecture of hardware-assisted virtualization (left) and how CloudVisor leverages it to secure control transfer using nested virtualization.

Dynamic Nested Virtualization

- CloudVisor does not contain machine bootstrap code for the sake of small TCB.
- Consequently, it is **booted after the VMM and the management VM have been initialized.**
- When CloudVisor boots, it runs in host mode and demotes the VMM to guest mode, thus effectively virtualizes the VMM on the fly.

MEMORY ISOLATION

Isolation with Nested/Extended Paging

general structure of extended paging

- The guest VM manages the address translation from VA to GPA with the conventional page table.
- translate guest physical addresses (GPA) to HPA using an extended page table (EPT)

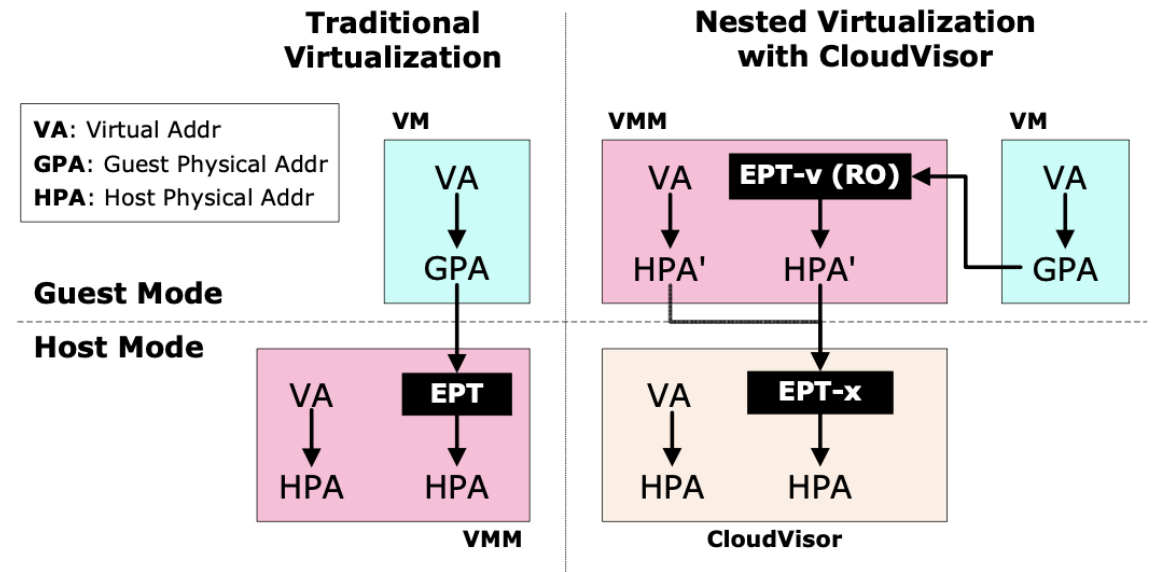


Figure 4: The general structure of extended paging (left) and how CloudVisor leverages it for memory isolation.

CloudVisor

- extended page table (EPT)
- CloudVisor: GPA-to-HPA mapping EPT for the VMM
- EPT-x is kept in the memory space of CloudVisor and is not accessible by the VMM.
- only supports platforms with hardware-assisted address translation.

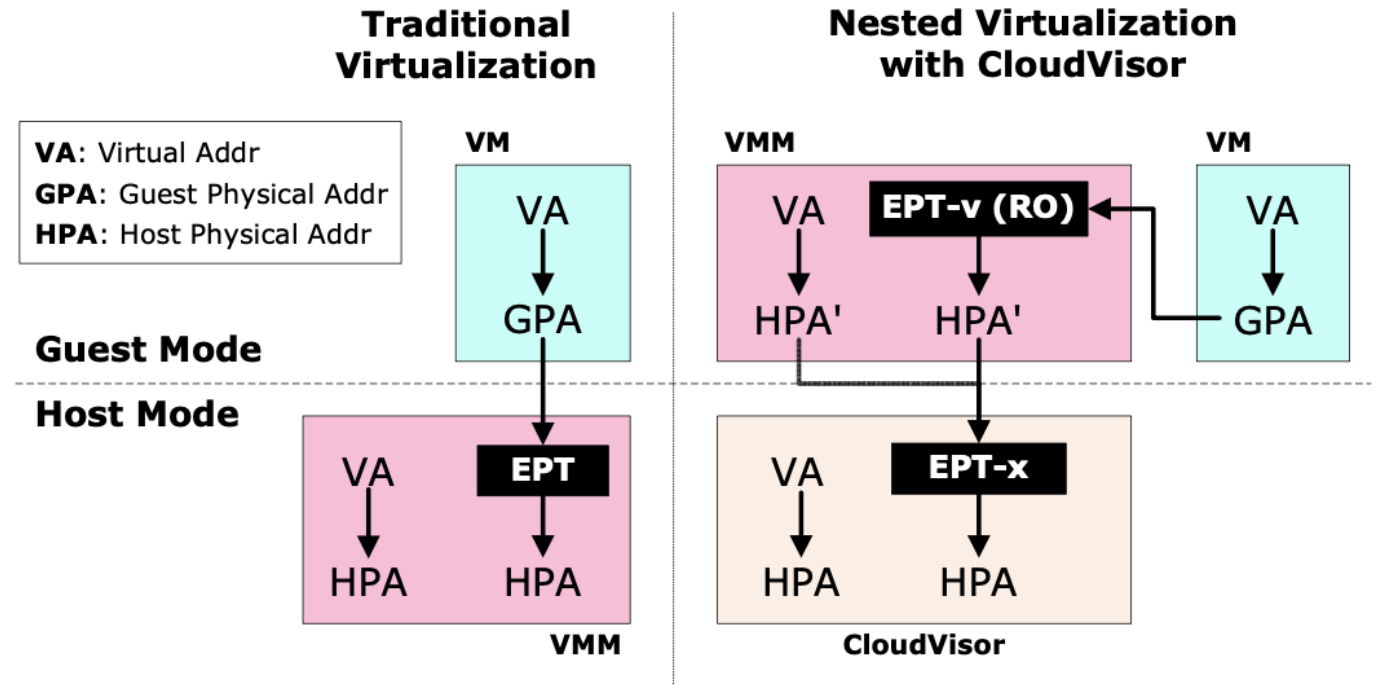


Figure 4: The general structure of extended paging (left) and how CloudVisor leverages it for memory isolation.

Memory Ownership Tracking

- maintains a table to track the ownership of each physical memory page.
- The value of the table is the owner ID of the page. Each VM is assigned with a unique ID when it is booted. The VMM's ID is fixed to zero.
- a physical memory page can only be assigned to be one owner at a time.

Legal Memory Accesses

CloudVisor interposes and assists such accesses to ensure that **only minimal insensitive information** will be divulged.

1. Privileged instructions such as I/O instructions and accesses to control registers cause traps (i.e., VM exits) that are handled by the VMM.
 - VMM needs to get the instruction opcode in the guest VM memory to emulate it
 - CloudVisor **only allows fetching one opcode** pointed by the program counter

Legal Memory Accesses

2. The VMM needs to walk the page table in the guest VM to translate the virtual addresses to guest physical addresses
 - CloudVisor temporarily allows the VMM to indirectly read the **guest page table entries** corresponding to **the opcode and memory operands**
 - CloudVisor fetches the program counter of the instruction and parses the instruction to get the memory operand.
 - CloudVisor walks the page table in the guest VM to get the page table entries required to translate the program counter and the memory operands
 - When the VMM accesses the page table, CloudVisor *feeds it with the previously obtained page table entries.*

Legal Memory Accesses

3. The VMM also needs to get the contents of guest I/O buffers when emulating I/O accesses.
 - When the VMM accesses I/O buffers, an EPT fault is raised and CloudVisor handles the fault by copying the data for the VMM.

DISK STORAGE PROTECTION

Two alternative ways

- The first one is letting a cloud user use an encrypted file system that guard the disk I/O data at the file-system level
- CloudVisor also provides full-disk encryption and hashing to protect disk data privacy and integrity. CloudVisor **encrypts the data exchange between a VM and the VMM** and verifies the integrity, freshness and ordering of disk I/O data.

Handling Data Exchange

- All data exchanged through disk I/O ports are encrypted and hashed before being copied to the VMM and decrypted before copying data to a guest VM. does not encrypt or decrypt data exchanges on other ports
- To determine if I/O data exchange between a guest VM and the VMM is legal, CloudVisor intercepts and parses I/O requests from the guest VM. CloudVisor retrieves the information of I/O port, memory address of the I/O buffer and the buffer size for further processing.

Handling Data Exchange

具体而言有两种主要的处理方式：

- “trap and emulate”. introduce a large amount of traps (i.e., VM exits) to CloudVisor if the data is copied in small chunks. 这种方案适合 port-based I/O
- using a bounce buffer in CloudVisor. 当产生数据交换的时候，CloudVisor会负责将数据存在buffer里面，VMM/guest VM再从buffer里面读。one additional data copy. 这种方案适合DMA

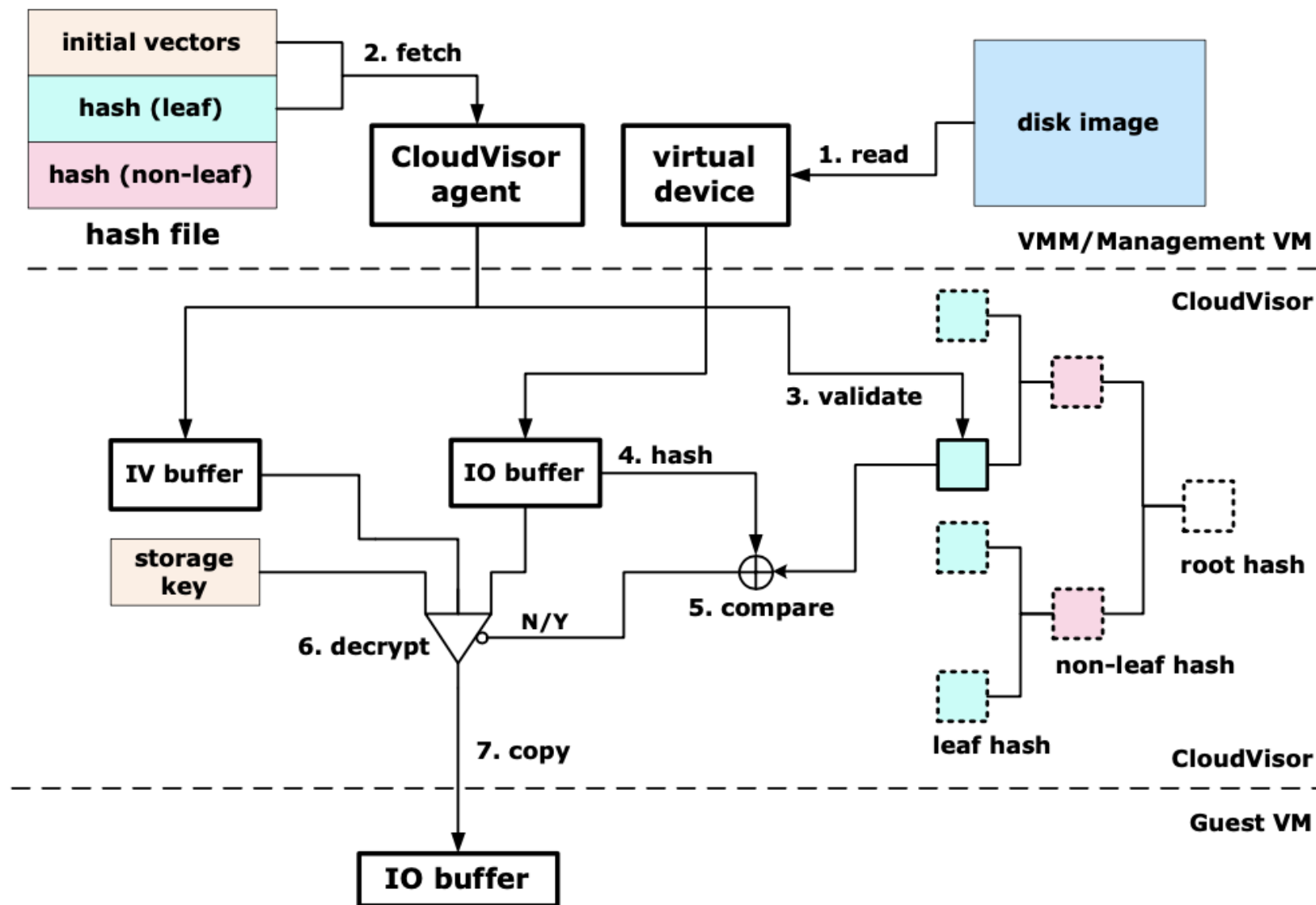
Disk I/O Privacy and Integrity

- uses the AES-CBC algorithm to encrypt and decrypt disk data in the granularity of disk sectors. A 128-bit AES key is generated by the user and passed to CloudVisor together with the encrypted VM image. The storage AES key is always maintained inside CloudVisor.
- At VM bootup time, CloudVisor fetches all non-leaf nodes of hashes and IVs (Initial Vectors) in the Merkle hash tree and keeps them as in-memory cache.
- 如何读：读之前先验证hash；如何写：为磁盘扇区生成IV（如果之前没有），hash，然后用密钥和IV加密。
- 使用不可信的用户级代理，协助获取文件的元数据。provide a user-level agent in the management VM to assist metadata fetching, updating and caching. The agent is untrusted as it has the same privilege as other software in the management VM

初始化流程

- for each disk sector, a 128-bit MD5 hash and a 128-bit IV are stored in a file stored in the file system of the management VM. The hash is organized using a Merkle tree to guard the freshness and ordering of disk data.
- When launching a guest VM, the file is mapped into the memory of the agent. (有点没懂) The agent fetches all the non-leaf hashes in the Merkle hash tree and sends them to CloudVisor. CloudVisor caches the hashes in memory to eliminate further fetches.

disk I/O read 流程



存在的问题

- The agent leverages the file cache of the operating system to buffer the most frequently used metadata. In the worst case, for a disk read, the agent **needs two more disk access** to fetch the corresponding hash and IV.
- **Sudden power loss may cause state inconsistency**, as CloudVisor currently does not guarantee atomic updates of disk data, hashes and IVs. For simplicity, CloudVisor assumes the cloud servers are equipped with power supply backups and can shutdown the machine without data loss on power loss.

IMPLEMENTATION ISSUES AND STATUS

- based on commercially available hardware support for virtualization, including VT-x, EPT, VT-d, and TXT
- To defend against cache-based side-channel attacks among VMs, CloudVisor uses the AES- NI instruction set in CPU to do encryption
- CloudVisor currently only supports hardware-assisted virtualization
- supports Xen with both Linux and Windows as the guest operating systems and can run multiple uniprocessor and multiprocessor VMs

Multiple VMs and Multicore Support

- CloudVisor maintains one VMCS for each CPU core used by the VMM.
- All CPU cores shares one EPT for the VMM
- CloudVisor serializes accesses from multiple cores to EPT-x.
- During startup, SINIT/SKINIT uses IPIs (Inter-processor Interrupt) to broadcast to all CPU cores and launch CloudVisor on all the cores simultaneously.

VM Life-cycle Management

VM construction and destruction

- construction: I/O data are transparently decrypted when it is copied to guest VM memory.
- destruction: the access right of memory pages of the VM is restored to the VMM transparently.

VM Life-cycle Management

VM snapshot, save, restore

- CloudVisor uses perpage encryption and hashing to protect memory snapshot. The hashes are organized as a Merkle tree and stored together with an array of IVs.
- save: CloudVisor would encrypt the requested page with the AES storage key and a newly generated IV. The VMM will get an encrypted memory image.
- restore: the VMM copies the encrypted memory image into guest memory space. CloudVisor finds the corresponding storage key of the VM, fetches the IVs and hashes of the memory image and decrypts the pages.

VM Life-cycle Management

VM migration

- similar to VM save and restore
- requires an additional key migration protocol
- CloudVisor on the migration source and destination platform needs to verify each other before migrating the storage key and root hash.

Performance Optimization

- Boosting I/O with hardware support
- Reducing unnecessary VM exits (On Intel platform)
 - replaces the VM read and VM write instructions with memory accesses to the VMCS

Key Management

- the key management scheme is orthogonal to the protection means in CloudVisor and can be easily replaced with a more complex scheme.

Soft and Hard Reset

面对soft和hard reset攻击时的方法：

- soft reset: An attacker might issue a soft reset that does not reset memory and try to read the memory content owned by a leased VM.
 - the reset will send a corresponding INIT signal to the processor, which will cause a VMX exit to CloudVisor.
 - CloudVisor will then scrub all memory owned by the leased VMs and self-destruct by scrubbing its own memory
- hard reset: CloudVisor simply post the hard rest signals to CPUs

PERFORMANCE EVALUATION

TODO

LIMITATION AND FUTURE WORK

- Enhancing Protection :
 - a malicious VMM might try to mislead a VM by even discarding I/O requests from a VM
 - let a VM or CloudVisor to proofcheck services by the VMM
 - (future)it might be feasible to implement CloudVisor in hardware or firmware

LIMITATION AND FUTURE WORK

- Impact on VMM's Functionality:
 - inhibit VM **introspection systems** and memory sharing systems for memory isolation and encryption
 - (future)allowing some pages being shared read-only among VMs, and validating any changes to these pages in CloudVisor.
 - a **fail-stop** approach against possible attacks or crashes can be replaced by a **fail-safe** approach to improving the reliability of VMs atop CloudVisor.

LIMITATION AND FUTURE WORK

- Supporting Other VMMs :
 - currently only tested CloudVisor for the Xen VMM
 - valuation of CloudVisor on other VMMs and OSes
- Verification :
 - formally verify its correctness and security properties

Thanks!